Problem 3: Warehouse Stock Ledger

KJBN Labs · Version 1.0 · October 2026 · Read together with the *Common Instructions and Evaluation Criteria* PDF

Before you start: the common PDF defines the deliverables (API + UI + database + hosting + unit tests with coverage report + prompt log), the recommended stacks, the milestone commit rules, the README and SUBMISSION.md formats, and the rubric. This PDF contains only the problem. Build the **Core** requirements fully and test them before touching any Stretch item. Every business rule below must be enforced in the backend. Record every ambiguity you resolve as a numbered assumption in your README.

PROBLEM 3

Warehouse Stock Ledger

Type: Immutable ledger / derived state / atomicity · Domain: Small warehouse

Context. A small distributor keeps stock in a warehouse with several bins. Stock quantities are currently edited by hand and nobody can explain why the system says 40 bags when the floor has 36. Build a system where stock is never edited — only *movements* are recorded, and every balance is explainable from them.

Entities

- **Product** — SKU (unique), name, unit (kg / pcs / litre), reorder level.
- **Location** — bin code (unique), description.
- **Movement** (append-only) — type (**IN**, **OUT**, **TRANSFER**, **ADJUSTMENT**), product, from_location / to_location as applicable, quantity, unit_cost (IN only), reason, reference (invoice no., order no.), created_at, created_by, reverses_movement_id (nullable).

Business rules

1. Movements are **immutable**: no update or delete endpoint exists. A mistake is corrected by posting a *reversal* movement that references the original; the original remains visible and marked as reversed.
2. Balance per (product, location) is **derived** from movements. You may cache it, but you must provide a *reconcile* endpoint that recomputes from scratch and reports any drift.
3. Stock at any location can never go negative. An OUT or TRANSFER that would do so is rejected, including under concurrent requests.
4. A TRANSFER moves quantity between two locations atomically — it either fully happens or not at all, and it appears as one logical movement in history.
5. Product valuation uses **weighted average cost**: every IN updates the product's average cost; OUTs are valued at the average cost at that moment, and that value is stored on the movement (so history does not change when the average changes later).
6. ADJUSTMENT requires a reason from a fixed list (damaged, expired, count correction, theft, other-with-note) and may be positive or negative.
7. A product whose total balance across all locations is at or below its reorder level appears in the low-stock report.

Core features

- **API:** manage products and locations; post each movement type; reverse a movement; balances (by product, by location, by both); movement history with filters; low-stock report; valuation report (quantity × average cost per product); reconcile.
- **UI:** stock overview grid (products × locations); a movement entry form per type with inline validation; product page with running balance history ("ledger view"); low-stock and valuation pages.

Edge cases we will try

Two OUTs racing for the last units; a TRANSFER where the destination write fails midway; reversing an already reversed movement; IN with zero or negative cost; rounding of average cost after many INs; reconcile after you deliberately corrupt the cache.

Stretch

Stock-take mode: enter counted quantities and auto-generate the adjustments; CSV export of ledger; batch/lot tracking with expiry.

> **Reminder of what wins points:** correct rules under bad input and concurrency; a clean separation of API, business rules and persistence; distinct error types with consistent responses; structured logs with context; boundary tests for every numeric rule; honest README and write-up. See the common PDF, Section 8.